

A range facility for `mdspan`

- - Abstract
 - Revision history
 - Discussion
 - Design
 - Implementation experience
 - Proposed change
 - Acknowledgements

Abstract

Standard `mdspan` accessors, such as `default_accessor`, are specialized for use with raw pointers. While `mdspan`'s design allows for custom accessors, providing a standardized iterator-based accessor simplifies integration with ranges such as views and containers.

This paper proposes `iterator_accessor`, which leverages `random_access_iterator` to decouple multi-dimensional views from physical memory continuity. Furthermore, it introduces new range-based constructors (`from_range_t`) for `mdspan` that enable overhead-free integration with any random-access range.

This new interface is both easier to use and much safer. Since a range knows its own size, `mdspan` can now automatically check if the dimensions fit, catching errors early at compile-time or during runtime. Additionally, using ranges instead of raw pointers prevents slicing bugs, where a pointer to a derived class is accidentally treated as a base class pointer, leading to memory corruption.

Revision history

R0

Initial revision.

Discussion

By default, `mdspan` provides accessors optimized for contiguous memory via raw pointers. Although designed as a general-purpose view, the standard `default_accessor` and `aligned_accessor` are strictly coupled with pointer-based data handles.

Take `default_accessor` as an example:

```
template<class ElementType>
struct default_accessor {
    using offset_policy = default_accessor;
    using element_type = ElementType;
    using reference = ElementType&;
    using data_handle_type = ElementType*;

    constexpr default_accessor() noexcept = default;
    template<class OtherElementType>
    constexpr default_accessor(default_accessor<OtherElementType>) noexcept;
    constexpr reference access(data_handle_type p, size_t i) const noexcept;
    constexpr data_handle_type offset(data_handle_type p, size_t i) const noexcept;
};
```

It performs two core operations:

- `access(p, i)`: Returns `p[i]`.
- `offset(p, i)`: Returns `p + i`.

These are the fundamental operations defined by the `random_access_iterator`. A random-access iterator models sequences that support constant-time indexing and arbitrary offsets. The fact that `default_accessor` mirrors these operations suggests that `mdspan` can support iterator-based data handles, making an iterator-based accessor a natural extension.

By introducing `iterator_accessor` as follows:

```
template<random_access_iterator I>
struct iterator_accessor {
    using offset_policy = iterator_accessor;
    using data_handle_type = I;
    using reference = iter_reference_t<I>;
    using element_type = remove_reference_t<reference>;

    constexpr iterator_accessor() noexcept = default;

    constexpr reference access(data_handle_type p, iter_difference_t<I> i) const
    { return p[i]; }
    constexpr data_handle_type offset(data_handle_type p, iter_difference_t<I> i) const
    { return p + i; }
};
```

Any type satisfying `random_access_iterator` can serve as the data handle. The standard `default_accessor<T>` is conceptually and functionally equivalent to `iterator_accessor<T*>`, which suggests that `default_accessor` is a specialization of a more general *iterator*-based model, rather than a fundamentally distinct *pointer*-based design.

This provides a standardized mechanism to bridge abstract data sequences with multi-dimensional indexing, eliminating the need to reinvent access logic.

Additionally, `iterator_accessor` enables `mdspan` to support ranges yielding proxy references, a capability not supported in pointer-based accessors.

With the following existing CTAD:

```
template<class MappingType, class AccessorType>
mdspan(const typename AccessorType::data_handle_type&, const MappingType&,
       const AccessorType&)
-> mdspan<typename AccessorType::element_type, typename MappingType::extents_type,
       typename MappingType::layout_type, AccessorType>;
```

`mdspan` can be instantiated with minimal boilerplate:

```
auto map_3x3 = layout_right::mapping(extents(3, 3));
auto r = {1, 2, 3, 4, 5, 6, 7, 8, 9};
auto ms = mdspan(r.begin(), map_3x3, iterator_accessor<decltype(r.begin())>{}); // mdspan<const int, ...>, equivalent to mdspan(l.begin(),
```

Additionally, this paper introduces the `from_range_t` constructor for `mdspan`. It allows users to construct `mdspan` directly from any `random_access_range` with significantly cleaner syntax:

Example 1: Integration with Non-contiguous Ranges

```
// helper function to simplify spelling of ranges::to<mdspan>
auto as_mdspan = [](auto... exts) { return ranges::to<mdspan>(exts...); };

auto grid_10x10 = views::iota(0) | as_mdspan(10, 10); // mdspan<const int, ...>

auto zeros = views::repeat(0);
auto zero_10x10 = zeros | as_mdspan(10, 10);           // mdspan<const int, ...>

auto reg = simd::vec<float>(0.42);
auto mat_2x2 = mdspan(from_range, reg, 2, 2);         // mdspan<const float, ...>
```

Example 2: Handling Proxy References

```
auto r = vector{true, false, true, false};
auto ms = mdspan(from_range, r, 2, 2);                 // mdspan<bool, ...>
auto cms = mdspan(from_range, views::as_const(r), 2, 2); // mdspan<const bool, ...>
```

Example 3: Compile-time and Runtime Size Validation

```
array<int, 12> arr{};
auto m1 = mdspan(from_range, arr, cw<3>, cw<4>);      // ok
auto m2 = mdspan(from_range, arr, cw<4>, cw<4>);      // compile-time error

vector<float> v(15);
auto m3 = mdspan(from_range, v, 3, 5);                 // ok
auto m4 = mdspan(from_range, v, 4, 5);                 // runtime assert
```

Example 4: Compile-time Array Slicing protection

```
struct Base {};
struct Derived : Base {};
Derived data{4};
auto m1 = mdspan<Base, dims<2>>(data, 2, 2);           // this is currently ok
auto m2 = mdspan<Base, dims<2>>(from_range, data, 2, 2); // compile-time error
```

Example 5: Integration with Range adaptors

```
vector<int> v1{1, 2, 3}, v2{4, 5}, v3{};
array a{6, 7, 8};
auto s = views::single(9);
auto r = views::concat(v1, v2, v3, a, s);
auto ms = mdspan(from_range, r, 3, 3);                 // mdspan<int, ...>

vector x_coords = {0, 10, 20};
vector y_coords = {0, 5};
vector z_coords = {0, 100};
// Represents a 3D grid of coordinate tuples: (x, y, z)
auto coord_grid = views::cartesian_product(x_coords, y_coords, z_coords);
auto points = coord_grid | as_mdspan(x_coords.size(), y_coords.size(), z_coords.size()); // mdspan<tuple<int, int, int>, ...>
auto [x, y, z] = points[1, 0, 1];
```

Example 6: Enabling Multi-dimensional Move Semantics (with [P3242](#))

```
auto r = v | views::as_rvalue;
auto ms = mdspan(from_range, r, 3, 3);
linalg::copy(ms, dst);
```

Example 7: Multi-dimensional Type Erasure for Contiguous Ranges

```
void as_1d_span(span<const int>);
as_1d_span(vector{1, 2, 3, 4});
as_1d_span(array {1, 2, 3, 4});
as_1d_span(views::single(42) | views::as_const); // this is currently ok

void as_2d_span(mdspan<const int, dims<2>>);
as_2d_span( {from_range, vector{1, 2, 3, 4}, 2, 2 } );
```

```
as_2d_span( {from_range, array {1, 2, 3, 4, 5, 6},          2, 3} );
as_2d_span( {from_range, views::single(42) | views::as_const, 1, 1} );
```

Example 8: SoA-to-AoS mapping via `views::zip`

```
vector pos_x = {0.0, 1.0, 2.0, 3.0};
vector pos_y = {0.0, 0.5, 1.0, 1.5};
vector mask = {1, 0, 1, 0};

auto positions = views::zip(pos_x, pos_y, mask);
auto ms = positions | as_mdspan(2, 2); // mdspan<tuple<double, double, int>, ...>
auto cms = positions | views::as_const | as_mdspan(2, 2); // mdspan<const tuple<double, double, int>, ...>

auto [x, _, active] = ms[1, 0];
if (active) {
    x += 10.0f;
}
```

Example 9: Abstracting Hardware Strides via `views::stride`

```
// Physical buffer: Data is interleaved or padded (e.g., 128-bit alignment)
vector<float> hardware_buffer = { /* large padded data */ };
auto ms = hardware_buffer | views::stride(4) | as_mdspan(4, 4); // mdspan<float, ...>
```

Design

Deduction of `element_type`

A critical requirement of `mdspan` is that its first template argument, `ElementType`, must match the accessor's `element_type`. Furthermore, `mdspan::value_type` is strictly defined as `remove_cv_t<element_type>`.

Earlier iterations of this design attempted to define `element_type` as `remove_reference_t<iter_reference_t<I>>`.

While this preserves `const`-qualification for contiguous iterators (e.g., `const float*` results in `const float`), it fails for proxy iterators. For instance, with `vector<bool>` or `views::zip`, this would force `element_type` to be the `vector<bool>::reference` or `tuple<double&, int&>`, which unnecessarily couples the `mdspan` type to the internal implementation of the accessor and breaks the expectation that `value_type` represents the underlying data.

The `element_type` should be defined as a `cv`-qualified version of `iter_value_t<I>`. This allows the accessor to return a proxy object through `operator[]` while letting `mdspan` maintain a clean, readable `element_type` such as plain `bool`.

The determination of `const`-qualification can be performed by the C++23 constant-iterator ([\[const.iterators.alias\]](#)) framework from [P2278](#). By utilizing the `constant_iterator` concept checking, the accessor can determine if the iterator is read-only, regardless of whether it returns a true reference or a proxy. If the iterator is a `constant_iterator`, `element_type` is deduced as `const iter_value_t<I>`; otherwise, it is simply `iter_value_t<I>`.

An alternative approach would be to use `indirectly_writable<I, iter_value_t<I>>` to detect mutability; However, this can be problematic for certain iterator adaptors. For instance, `move_iterator` is generally not `indirectly_writable<int>` because its `operator*` returns an rvalue reference, which cannot be the target of an assignment:

```
vector v = {1, 2, 3};
move_iterator it(v.begin());
*it = 42; // error: using rvalue as lvalue
```

In this case, `indirectly_writable` would incorrectly categorize `move_iterator` as a constant iterator, forcing `element_type` to be `const int`. Therefore, the author prefers using the `constant_iterator` concept as it more accurately captures the semantic intent of read-only access.

For proxy iterators, we only consider `const`-qualification and do not attempt to propagate `volatile`. This is a deliberate simplification, as `volatile` is generally less common in high-level proxy scenarios compared to `const`. Furthermore, it is non-trivial to determine a canonical `element_type` when the underlying range returns complex proxy reference, such as `tuple<volatile int&, int&, const int&>`.

Support for Conversions

A key design principle for accessor is the strict mirroring of conversion semantics from the underlying data handle, for example, from `iterator` to `const_iterator`. The relationship between `iterator_accessor<I>` and `iterator_accessor<I2>` must faithfully reflect the relationship between the iterators `I` and `I2`.

If an iterator `I` is constructible from `I2`, but `I2` is not implicitly convertible to `I`, the accessor must maintain this exact distinction. To align with the design intent of `mdspan`, the conversion constructor can be defined as:

```
template<typename I2>
requires is_constructible_v<I, I2>
explicit(!is_convertible_v<I2, I>)
constexpr iterator_accessor(iterator_accessor<I2>) noexcept {}
```

By using `explicit(!is_convertible_v<I2, I>)`, the accessor correctly inherits the implicitness of the underlying iterator. This ensures that `mdspan` conversion works exactly as the user expects based on the behavior of their chosen iterator type. These conversions must also be constrained to prevent unsafe pointer-to-derived to pointer-to-base transitions that would invalidate multidimensional indexing.

It is reasonable for `iterator_accessor` to support construction from a `default_accessor`. This reflects the standard behavior where various iterator adaptors can be naturally initialized from raw pointers. This allows a pointer-based `mdspan` to be used as a source for a range-aware `mdspan` without friction:

```
int arr[] = {1, 2, 3, 4, 5, 6};
mdspan legacy_ms(arr, 2, 3);
mdspan<int, dims<2>, layout_right,
    iterator_accessor<move_iterator<int*>>> move_ms(legacy_ms); // ok, default_accessor -> iterator_accessor
mdspan<int, dims<2>, layout_right,
    iterator_accessor<reverse_iterator<move_iterator<int*>>>> r_move_ms(move_ms); // ok, iterator_accessor -> iterator_accessor
```

It is also meaningful to provide conversion operators from `iterator_accessor` to `default_accessor`, as certain contiguous iterators, such as `basic_const_iterator<int*>` can convert to a `const int*` pointer. If a contiguous iterator can already be unwrapped into a pointer, preventing the `mdspan` itself from doing so would create an unnecessary and inconsistent restriction.

Offset Type in access/offset

According to the [\[mdspan.accessor\]](#) requirements, those two functions take a `size_t` offset, as the accessor is intended to be decoupled from and unaware of the specific `index_type` used by the layout mapping.

The design adheres to this by using `size_t` at the interface and performing an internal `static_cast<iter_difference_t<I>>(i)`.

This is necessary as iterator models are only guaranteed to support arithmetic with their `difference_type`. Furthermore, we require that `i` be representable in `difference_type` as a precondition, guaranteeing that the mapping remains well-defined.

New Range Constructor for mdspan

If only `iterator_accessor` were introduced, the most direct way to create a iterator-based `mdspan` would remain verbose with spelling `r.begin()` twice:

```
auto map_3x3 = layout_right::mapping(extents(3, 3));
auto ms = mdspan(r.begin(), map_3x3, iterator_accessor<decltype(r.begin())>{});
```

Instead, we provide a clearer, range-aware interface as follows (we do not currently provide overloads of `array` or `span` as extents, to keep the interface minimal):

```
template<ranges::random_access_range R, class... OtherIndexTypes>
requires ranges::borrowed_range<R>
constexpr mdspan(from_range_t, R& r, OtherIndexTypes... exts)
: ptr_(ranges::begin(r)), ...

template<ranges::random_access_range R, class... Integrals>
requires (is_convertible_v<Integrals, size_t> && ...)
mdspan(from_range_t, R& r, Integrals...)
-> mdspan<typename iterator_accessor<ranges::iterator_t<R>>::element_type,
    extents<size_t, maybe_static_ext<Integrals>...>,
    layout_right,
    iterator_accessor<ranges::iterator_t<R>>>
```

Above, the requirement for `borrowed_range` is a safety guard: it ensures the `mdspan` does not outlive its underlying storage. The constructor initializes the data handle directly via `ranges::begin(r)`. This allows users to spell it out simply with:

```
auto ms = mdspan(from_range, r, 3, 3);
```

Or via `ranges::to`, as it natively supports `from_range_t` tag-dispatched constructors according to [\[range.utility.conv.to\]](#):

```
auto ms = r | ranges::to<mdspan>(3, 3);
```

It is worth noting that `ranges::to<mdspan>` is already supported under the current standard for raw arrays. For instance, the following is already [well-formed](#) based on [\[range.utility.conv.to\]](#):

```
int arr[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
auto ms_r0 = arr | ranges::to<mdspan>(); // mdspan<int, extents<size_t, 12>, ...>
auto ms_r1_12 = arr | ranges::to<mdspan>(12); // mdspan<int, extents<size_t, dynamic_extent>, ...>
auto ms_r2_3x4 = arr | ranges::to<mdspan>(3, 4); // mdspan<int, extents<size_t, dynamic_extent, dynamic_extent>, ...>
auto ms_r3_2x2x3 = arr | ranges::to<mdspan>(2, 2, 3); // mdspan<int, extents<size_t, dynamic_extent, dynamic_extent, dynamic_extent>
```

Since `ranges::to` attempts direct construction `mdspan(arr, exts...)`, which happens to match `mdspan`'s pointer-based constructor via implicit decay.

(While some might find the use of `ranges::to` to produce a view-like object unusual, the author maintains that this is a natural and highly desirable pattern, as detailed in [P3544](#) (`ranges::to<view>`), but that's another topic)

By adding the `from_range_t` constructor, we are generalizing this intuitive pattern to all random access ranges. It ensures a consistent interface for making `mdspan` regardless of whether the underlying storage is a raw array, a standard container, or a sophisticated range-based view:

```
int arr1[] = {1, 2, 3, 4};
auto ms1 = arr1 | ranges::to<mdspan>(2, 2); // ok before this proposal
array arr2 = {1, 2, 3, 4};
auto ms2 = arr2 | ranges::to<mdspan>(2, 2); // ok after this proposal
```

The `from_range` tag serves as an explicit *opt-in*, signaling that user intends to use the range as the bottom layer of the `mdspan` rather than the raw pointer, which improves the usability of complex views such as:

```
vector x_coords = {0, 10, 20};
vector y_coords = {0, 5};
vector z_coords = {0, 100};
auto coord_grid = views::cartesian_product(x_coords, y_coords, z_coords);
auto ms1 = mdspan(from_range, coord_grid, x_coords.size(), y_coords.size(), z_coords.size());

vector pos_x = {0.0, 1.0, 2.0, 3.0};
vector pos_y = {0.0, 0.5, 1.0, 1.5};
vector mask = {1, 0, 1, 0};
auto ms2 = mdspan(from_range, views::zip(pos_x, pos_y, mask), 2, 2);
```

A significant safety advantage of this constructor is that it formalizes bounds validation within the standard. While an implementation of `default_accessor` or `aligned_accessor` could technically perform validation, the current standard does not require it.

When `R` models `sized_range`, it enables `mdspan` to validate that the total number of elements required by the extents, providing a layer of runtime or even compile-time protection:

```
array<int, 5> r{};
auto ms1 = mdspan(from_range, r, cw<2>, cw<3>); // compile-time error
```

```
auto ms2 = r | as_mdspan(cw<2>, cw<3>); // compile-time error
```

This is because when using *integral-constant-like* types for extents, the mapping's `required_span_size()` becomes a constant expression. If the source range also provides a static size, such as `array` or `span` with a static extent, the constructor can trigger a `static_assert` to catch the overflow at compile-time. For dynamic ranges like `vector`, this check gracefully drops down to a hardened precondition.

Beyond size validation, this constructor provides a unique layer of protection against slicing, which pointer-based constructors cannot catch: when a pointer to a `Derived` array is passed to an `mdspan<Base>`, the compiler implicitly converts the `Derived*` to a `Base*` at the call site (cf. LWG [4405](#)). This causes internal stride calculations to be incorrect, leading to a dangerous out-of-bounds access:

```
struct Base {};
struct Derived : Base {};
Derived data[4];
auto m1 = mdspan<Base, dims<2>>(data, 2, 2); // accepts but wrong
```

By contrast, `from_range_t` captures the entire range type. This allows the constructor to see the original `Derived` type and perform a compile-time check:

```
auto m2 = mdspan<Base, dims<2>>(from_range, data, 2, 2); // compile-time error
```

Optimizing for Contiguous Ranges

A potential concern with introducing `iterator_accessor` is the instantiation bloat caused by creating unique accessor types for every distinct iterator. To address this, we propose that the `from_range_t` constructor and its associated CTAD prefer `default_accessor` whenever the source models `contiguous_range`.

To achieve this optimization, the `mdspan` constructor initializes its data handle using `ranges::data(r)` rather than `ranges::begin(r)`, mirroring the behavior of `span`:

```
template<ranges::random_access_range R, class... OtherIndexTypes>
constexpr mdspan(from_range_t, R&& r, OtherIndexTypes... exts)
: ptr_(
    /* Case 1: contiguous_range -> ranges::data(r) */
    /* Case 2: non-contiguous    -> ranges::begin(r) */
), ...

template<ranges::random_access_range R, class... Integrals>
mdspan(from_range_t, R&&, Integrals...)
-> mdspan</* element_type *//,
    /* extents *//,
    layout_right,
    /* Case 1: contiguous_range -> default_accessor */
    /* Case 2: non-contiguous    -> iterator_accessor *//>;
```

This approach aligns with the direction of the standard libraries, particularly [P3349](#), which advocates that libraries are free to treat contiguous iterators as raw pointers to improve efficiency. We also see this pattern established in `views::counted`, which returns a `span` when given a contiguous iterator.

By following this precedent, `from_range_t` allows `mdspan` to function as a powerful multi-dimensional type eraser similar to `span`, which provides a consistent interface while still maintaining the high performance and high-level safety, as the constructor has performed the necessary checks:

```
void as_1d_span(span<const int>);

as_1d_span(vector{1, 2, 3, 4});
as_1d_span(array {1, 2, 3, 4});
as_1d_span(views::single(42) | views::as_const); // this is currently ok

void as_2d_span(mdspan<const int, dims<2>>);

as_2d_span( {from_range, vector{1, 2, 3, 4}, 2, 2} );
as_2d_span( {from_range, array {1, 2, 3, 4, 5, 6}, 2, 3} );
as_2d_span( {from_range, views::single(42) | views::as_const, 1, 1} ); // ok after this proposal
```

Support for Explicit Layout policy

While the proposed `from_range_t` constructor simplifies the creation of `mdspan`, it is currently difficult to specify a non-default layout when deal with ranges.

This is because the initial set of constructors does not offer a dedicated path for layout customization, which forces users to manually spell out all template parameters of `mdspan` even if they only wish to change the default layout. Furthermore, having to explicitly specify the iterator type for `iterator_accessor` is not only intrusive but also verbose and error-prone, as it negates the primary benefits of CTAD.

This creates a significant ergonomic barrier: developers cannot easily leverage CTAD to deduce the `element_type` and `extents_type` while simultaneously selecting a different layout, such as `layout_left` or `layout_stride`. To resolve this, we propose an additional constructor overload and a corresponding deduction guide that allows `mdspan` to be initialized directly from a pre-configured mapping object:

```
template<class R, class MappingType>
mdspan(from_range_t, R&&, const MappingType&)
-> mdspan</* element_type *//,
    typename MappingType::extents_type,
    typename MappingType::layout_type>,
    /* Case 1: contiguous_range -> default_accessor */
    /* Case 2: non-contiguous    -> iterator_accessor *//>;
```

This enables users to write expressive and concise code, such as:

```
auto ms1 = mdspan(from_range, v, layout_stride ::mapping(extents(8, 8), array{3, 5}));
auto ms2 = mdspan(from_range, v, layout_left_padded<2>::mapping(extents(6, 6)));
auto ms3 = v | as_mdspan(layout_left ::mapping(extents(4, 4)));
```

where the `mdspan` correctly deduces its element type from the range and its layout from the provided mapping, all while maintaining the safety and optimization benefits of the `from_range_t` interface.

CTAD for statically sized ranges in the 0-rank case

The current CTAD behavior of `mdspan` in the 0-rank case is largely a result of the design introduced by [P2554](#), which deliberately added two deduction paths for `mdspan`:

```
template<class CArray>
requires (is_array_v<CArray> && rank_v<CArray> == 1)
mdspan(CArray&)
-> mdspan<remove_all_extents_t<CArray>, extents<size_t, extent_v<CArray, 0>>>;

template<class Pointer>
requires (is_pointer_v<remove_reference_t<Pointer>>)
mdspan(Pointer&&)
-> mdspan<remove_pointer_t<remove_reference_t<Pointer>>, extents<size_t>>;
```

As a consequence, constructing `mdspan` from a raw array preserves the array bound, while constructing it via the pointer produces a rank-0 `mdspan` with no extents:

```
int x[5] = {1, 2, 3, 4, 5};
mdspan ms1{ x }; // mdspan<int, extents<size_t, 5>>, ...>
mdspan ms2{ auto(x) }; // mdspan<int, extents<size_t> >>, ...>
```

This proposal applies a similar deduction model to ranges, which is solely based on whether the size of the range is available as a constant expression. If the range size is known at compile time, it is reasonable to reflect that in the type and deduce `extents<size_t, ranges::size(r)>`; otherwise, the deduction naturally falls back to a rank-0 `mdspan`:

```
int x[5] = {1, 2, 3, 4, 5};
mdspan ms1{from_range, x}; // mdspan<int, extents<size_t, 5>>, ...>

vector v{1, 2, 3, 4, 5};
mdspan ms2{from_range, v}; // mdspan<int, extents<size_t> >>, ...>

auto s = views::single(42);
mdspan ms3{from_range, s}; // mdspan<int, extents<size_t, 1>>, ...>

auto e = views::empty<int>;
mdspan ms4{from_range, e}; // mdspan<int, extents<size_t, 0>>, ...>

optional o{42};
mdspan ms5{from_range, o}; // mdspan<int, extents<size_t> >>, ...>

o.reset();
mdspan ms6{from_range, o}; // mdspan<int, extents<size_t> >>, ...>, runtime assert
```

Implementation experience

The author implemented `iterator_accessor` and `from_range_t` constructor based on `libstdc++` along with the above example.

See [godbolt link](#) for details.

Proposed change

This wording is relative to [Latest Working Draft](#).

1. Modify 21.4.1 17.3.2 [\[version.syn\]](#) as indicated:

```
#define __cpp_lib_mdspan YYYYMMML // freestanding, also in <mdspan>
```

2. Modify 21.4.1 [\[mdspan.syn\]](#) as indicated:

```
// mostly freestanding
namespace std {
[...]
```

// [mdspan.accessor.aligned], class template aligned_accessor
template<class ElementType, size_t ByteAlignment>
class aligned_accessor;

// [mdspan.accessor.iter], class template iterator_accessor
template<random access iterator I>
class iterator_accessor;

```
[...]
```

3. Add `[mdspan.accessor.iter]` after [\[mdspan.accessor.aligned\]](#) as indicated:

[23.7 Overview \[mdspan.accessor.iter.overview\]](#)

```
namespace std {
  template<random access iterator I>
  struct iterator_accessor {
    using offset_policy = iterator_accessor;
    using data_handle_type = I;
    using element_type = see below;
    using reference = iter_reference_t<I>;

    constexpr iterator_accessor() noexcept = default;

    template<class I2>
    constexpr explicit(see below) iterator_accessor(iterator_accessor<I2>) noexcept;
    template<class ElementType>
    constexpr explicit(see below) iterator_accessor(default_accessor<ElementType>) noexcept;
    template<class ElementType>
    constexpr operator default_accessor<ElementType>() const noexcept;

    constexpr reference access(data_handle_type p, size_t i) const;
```

```
constexpr data_handle_type offset(data_handle_type p, size_t i) const;
};
}
```

-1- iterator_accessor meets the accessor policy requirements.

-2- element_type is required to be a complete object type that is neither an abstract class type nor an array type.

-3- Each specialization of iterator_accessor is a trivially copyable type that models semiregular.

-4- $[0, n)$ is an accessible range for an object p of type data_handle_type and an object of type iterator_accessor if and only if $[p.p + n)$ is a valid range.

-5- The member typedef-name element_type is defined as follows:

(5.1)- If I models contiguous iterator, then remove reference t<iter reference t<I>>.

(5.2)- Otherwise, if I models constant-iterator ([\(const.iterators.alias\)](#)), then const_iter_value t<I>.

(5.3)- Otherwise, element_type denotes iter_value t<I>.

23.7 Members [\[mdspan.accessor.iter.members\]](#)

```
template<class I2>
constexpr explicit(see below) iterator_accessor(iterator_accessor<I2>) noexcept;
```

-1- Constraints: Let $E2$ be typename iterator_accessor<I2>::element_type.

(1.1)- is constructible v<I, I2> is true.

(1.2) - I does not satisfy contiguous iterator, or $I2$ does not satisfy contiguous iterator, or is convertible v<E2(*)[], element_type(*)[]> is true.

-2- Effects: None.

-3- Remarks: The expression inside explicit is equivalent to:

!is convertible v<I2, I>

```
template<class OtherElementType>
constexpr explicit(see below) iterator_accessor(default_accessor<OtherElementType>) noexcept;
```

-4- Constraints:

(4.1)- is constructible v<I, OtherElementType*> is true.

(4.2)- I does not satisfy contiguous iterator or is convertible v<OtherElementType(*)[], element_type(*)[]> is true.

-5- Effects: None.

-6- Remarks: The expression inside explicit is equivalent to:

!is convertible v<OtherElementType*, I>

```
template<class ElementType>
constexpr operator default_accessor<ElementType>() const noexcept;
```

-7- Constraints:

(7.1)- is convertible v<I, OtherElementType*> is true.

(7.2)- I does not satisfy contiguous iterator or is convertible v<element_type(*)[], OtherElementType(*)[]> is true.

-8- Effects: Equivalent to: return {};

```
constexpr reference access(data_handle_type p, size_t i) const;
```

-9- Preconditions: i is representable as a value of type iter_difference t<I>.

-10- Effects: Equivalent to: return p[static_cast<iter_difference t<I>>(i)];

```
constexpr data_handle_type offset(data_handle_type p, size_t i) const;
```

-11- Preconditions: i is representable as a value of type iter_difference t<I>.

-12- Effects: Equivalent to: return p + static_cast<iter_difference t<I>>(i);

4. Modify 23.7.3.6.1 [\[mdspan.mdspan.overview\]](#) as indicated:

```
namespace std {
    template<class ElementType, class Extents, class LayoutPolicy = layout_right,
            class AccessorPolicy = default_accessor<ElementType>>
    class mdspan {
    public:
        [...]
        template<class R, class... OtherIndexTypes>
        constexpr mdspan(from_range_t, R&& r, OtherIndexTypes... exts);

        template<class... OtherIndexTypes>
        constexpr explicit mdspan(data_handle_type ptr, OtherIndexTypes... exts);
    };
}
```



```

[...]  

template<class R>  

constexpr mdspan(from_range_t, R&& r, const mapping_type& m);  

  

constexpr mdspan(data_handle_type p, const mapping_type& m);  

[...]  

};  

[...]  

  

template<class R>  

mdspan(from_range_t, R&& r) -> see below;  

  

template<class R, class... Integrals>  

mdspan(from_range_t, R&&, Integrals...) -> see below;  

  

template<class ElementType, class... Integrals>  

requires ((is_convertible_v<Integrals, size_t> && ...) && sizeof...(Integrals) > 0)  

explicit mdspan(ElementType*, Integrals...)  

-> mdspan<ElementType, extents<size_t, maybe-static-ext<Integrals>...>>;  

  

[...]  

template<class R, class MappingType>  

mdspan(from_range_t, R&&, const MappingType&) -> see below;  

  

template<class ElementType, class MappingType>  

mdspan(ElementType*, const MappingType&)  

-> mdspan<ElementType, typename MappingType::extents_type,  

    typename MappingType::layout_type>;  

[...]  

}

```

5. Add [mdspan.deduct] after [\[mdspan.mdspan.overview\]](#) as indicated:

23.7 Deduction guides

```

template<class R>  

mdspan(from_range_t, R&& r) -> see below;

```

-1- Constraints: R satisfies `ranges::random_access_range`.

-2- Let Accessor be `iterator accessor<ranges::iterator t<R>>`, let ElementType be `typename Accessor::element_type`, and let Extents be `extents<size_t, static cast<size_t>(ranges::size(r))>` if R models `ranges::sized_range` and `ranges::size(r)` is a constant expression, and `extents<size_t>` otherwise.

-3- Remarks: The deduced type is

(3.1) - `mdspan<ElementType, Extents>` if R satisfies `ranges::contiguous_range`,

(3.2) - `mdspan<ElementType, Extents, layout_right, Accessor>` otherwise.

```

template<class R, class... Integrals>  

mdspan(from_range_t, R&&, Integrals...) -> see below;

```

-4- Constraints:

(4.1) - R satisfies `ranges::random_access_range`

(4.2) - `(is_convertible_v<Integrals, size_t> && ...)` is true, and

(4.3) - `sizeof...(Integrals) > 0` is true.

-5- Let Accessor be `iterator accessor<ranges::iterator t<R>>`, let ElementType be `typename Accessor::element_type`, and let Extents be `extents<size_t, maybe-static-ext<Integrals>...>`.

-6- Remarks: The deduced type is

(6.1) - `mdspan<ElementType, Extents>` if R satisfies `ranges::contiguous_range`,

(6.2) - `mdspan<ElementType, Extents, layout_right, Accessor>` otherwise.

```

template<class R, class MappingType>  

mdspan(from_range_t, R&&, const MappingType&) -> see below;

```

-7- Constraints:

(7.1) - R satisfies `ranges::random_access_range`.

(7.2) - the *qualified-id* `MappingType::extents_type` is valid and denotes a type, and

(7.3) - the *qualified-id* `MappingType::layout_type` is valid and denotes a type.

-8- Let Accessor be `iterator accessor<ranges::iterator t<R>>` and let ElementType be `typename Accessor::element_type`.

-9- Remarks: The deduced type is

(9.1) - `mdspan<ElementType, typename MappingType::extents_type, typename MappingType::layout_type>` if R satisfies `ranges::contiguous_range`,

(9.2) - `mdspan<ElementType, typename MappingType::extents_type, typename MappingType::layout_type, Accessor>` otherwise.

6. Modify 23.7.3.6.2 [\[mdspan.mdspan.cons\]](#) as indicated:

```
template<class R, class... OtherIndexTypes>
constexpr mdspan(from range t, R&& r, OtherIndexTypes... exts);
```

-?- Let N be `sizeof...(OtherIndexTypes)`, let U be `remove reference t<ranges::range reference t<R>>`, and let I be $U*$ if R satisfies `ranges::contiguous_range`, and `ranges::iterator t<R>` otherwise.

-?- Constraints:

(2.1) - R satisfies `ranges::random_access_range`.

(2.2) - either R satisfies `ranges::borrowed_range` or is `const v<element type> && contiguous_iterator<data handle type> && contiguous_iterator<I>` is true.

(2.3) - `(is_convertible v<OtherIndexTypes, index type> && ...)` is true.

(2.4) - `(is_nothrow_constructible<index type, OtherIndexTypes> && ...)` is true.

(2.5) - $N == \text{rank}() \vee N == \text{rank_dynamic}()$ is true.

(2.6) - `is_constructible v<data handle type, I>` is true.

(2.7) - `is_constructible v<mapping_type, extents type>` is true.

(2.8) - `is_default_constructible v<accessor type>` is true, and

(2.9) - `contiguous_iterator<data handle type> && contiguous_iterator<I> && !is_convertible v<U(*)[], element type(*)[]>` is false.

-?- Mandates: If R models `ranges::sized_range`, `ranges::size(r)` is a constant expression, and `map.required_span_size()` is a constant expression, `ranges::size(r) ≥ map.required_span_size()` for the value of `map` after the invocation of this constructor.

-?- Preconditions:

(2.1) - R models `ranges::random_access_range`.

(2.2) - If `is const v<element type> && contiguous_iterator<data handle type> && contiguous_iterator<I>` is false, R models `ranges::borrowed_range`.

(2.3) - `[0, map.required_span_size())` is an accessible range of `ptr` and `acc` for values of `ptr`, `map`, and `acc` after the invocation of this constructor.

-?- Hardened preconditions: If R models `ranges::sized_range`, `ranges::size(r) ≥ map.required_span_size()` for the value of `map` after the invocation of this constructor.

-?- Effects:

(2.1) - `Direct-non-list-initializes ptr` with `ranges::data(r)` if R models `ranges::contiguous_range`, and `ranges::begin(r)` otherwise.,

(2.2) - `direct-non-list-initializes map` with `extents type(static_cast<index type>(std::move(exts))...)`, and

(2.3) - `value-initializes acc`.

[...]

```
template<class R>
constexpr mdspan(from range t, R&& r, const mapping_type& m);
```

-?- Let U be `remove reference t<ranges::range reference t<R>>` and let I be $U*$ if R satisfies `ranges::contiguous_range`, and `ranges::iterator t<R>` otherwise.

-?- Constraints:

(2.1) - R satisfies `ranges::random_access_range`.

(2.2) - either R satisfies `ranges::borrowed_range` or is `const v<element type> && contiguous_iterator<data handle type> && contiguous_iterator<I>` is true.

(2.3) - `is_constructible v<data handle type, I>` is true.

(2.4) - `is_default_constructible v<accessor type>` is true, and

(2.5) - `contiguous_iterator<data handle type> && contiguous_iterator<I> && !is_convertible v<U(*)[], element type(*)[]>` is false.

-?- Mandates: If R models `ranges::sized_range`, `ranges::size(r)` is a constant expression, and `m.required_span_size()` is a constant expression, `ranges::size(r) ≥ m.required_span_size()`.

-?- Preconditions:

(2.1) - R models `ranges::random_access_range`.

(2.2) - If `is const v<element type> && contiguous_iterator<data handle type> && contiguous_iterator<I>` is false, R models `ranges::borrowed_range`.

(2.3) - $[0, m.\text{required_span_size}())$ is an accessible range of ptr and acc for values of ptr and acc after the invocation of this constructor.

?- Hardened preconditions: If R models $\text{ranges}::\text{sized_range}$, $\text{ranges}::\text{size}(r) \geq m.\text{required_span_size}()$.

?- Effects:

(2.1) - Direct-non-list-initializes ptr with $\text{ranges}::\text{data}(r)$ if R models $\text{ranges}::\text{contiguous_range}$, and $\text{ranges}::\text{begin}(r)$ otherwise.,

(2.2) - direct-non-list-initializes map with m , and

(2.3) - value-initializes acc .

Acknowledgements

The author thanks Arthur O'Dwyer, Tomasz Kamiński, and Mark Hoemmen for their valuable feedback and insights, as always.